

University Transport

University Transport Management and Billing System



Session: 2025-Fall

Submitted By:

Ali Haider (2025-AI-515)

Supervised By:

Miss Namra Shiekh

Institute of Artificial Intelligence, Machine Learning and Cybersecurity

University of Engineering and Technology,

Lahore, (NC)



University Transport Management and Billing System

(Session 2025-Fall Artificial Intelligence)

The technical report is to be submitted to the Institute of Artificial Intelligence, Machine Learning and Cybersecurity, University of Engineering and Technology, Lahore (NC), for partial fulfillment of the requirements for a Bachelor of Science in Artificial Intelligence.

Supervisor:

Name:

Miss Namra Shiekh

Designation:

Associate Professor

Signature:

Chairman:

Signature:

Dean:

Signature:

Institute of Artificial Intelligence, Machine Learning and Cybersecurity

University of Engineering and Technology,

Lahore, (NC)



Declaration

I declare that the work contained in this technical report is my own, except where explicitly stated otherwise. Additionally, this work has not been submitted to obtain another degree or professional qualification.

Signature:

Dated:



Acknowledgments

First and foremost, we must express our gratitude to ALLAH Almighty and dedicate this endeavor to Him. We thank Him for the intellect bestowed upon us, the strength and serenity of our minds, and the good health that enabled us to complete this research.

We are deeply grateful to our parents for their unwavering support, encouragement, and sacrifices, which have been our greatest strength. We are profoundly obligated to the **University of Engineering and Technology Lahore, New campus** for their guidance and assistance in completing this research.

We would like to extend our special thanks and gratitude to our supervisor, **Miss Namra Shiekh**, for this innovative project idea and for imparting her wisdom and expertise during this research.

Additionally, we extend sincere appreciation to our friends and fellow researchers for their companionship, encouragement, and invaluable insights, all of which greatly enriched our research journey. Furthermore, we express our sincere gratitude to everyone who contributed to this research project, whether through assistance, feedback, or encouragement. Your support has been invaluable in our journey towards completion.



Table of Contents

Declaration	i
Acknowledgments	ii
List of Figures	vi
List of Tables	vii
Acronyms	viii
Abstract	ix
1. Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Objectives	1
2 Requirements Specification	2
2.1 Use Case Overview	2
2.2 Functional Requirements	2
2.3 Non-Functional Requirements	3
3 System Architecture	4
3.1 Architectural Overview	4
3.2 Technology Stack	4
3.3 Database Schema (File-Based)	5
3.4 Registration and Approval Workflow	5
4. Class Descriptions	7
4.1 User (Abstract Base Class)	8
4.2 Student	8
4.3 Admin	8
4.4 Vehicle (Abstract Base Class)	8
4.5 Bus and Van	9
4.6 Route	9
4.7 TransportPass and Bill (Composition Pair)	9
4.8 DataStore<T> (Template Class)	9
4.9 FileManager (Static Utility)	10



4.10 Report (Static Utility).....	10
5. Object Relationships	11
5.1 Composition: <i>TransportPass</i> and <i>Bill</i>	11
5.2 Aggregation: <i>main.cpp(Transport Manager)</i> and <i>Vehicle / Route</i>	11
5.3 Association: <i>Student</i> and <i>TransportPass</i>	12
6. UML Class Diagram	13
7. OOP Concepts Implemented.....	14
7.1 Abstract Classes and Virtual Functions.....	15
7.2 Inheritance.....	15
7.3 Template Class: <i>DataStore<T></i>	15
7.4 Dynamic Memory Management	15
7.5 Operator Overloading	16
7.6 Access Specifiers and Encapsulation	16
8. File Handling and Data Persistence	17
8.1 Serialize / Deserialize Pattern	17
8.2 Polymorphic Reconstruction.....	17
9. Menu Structure and Interaction Flow	18
9.1 Main Menu.....	18
9.2 Student Menu	18
9.3 Admin Menu	18
10. Testing and Evaluation.....	20
10.1 Functional Test Results	20
10.2 Build and Compilation.....	20
10.3 Recommended Demo Sequence for Screenshots.....	21
11. Code Quality and Project Statistics.....	22
11.1 Source File Summary	22
11.2 Coding Standards	23
12. Conclusion	24
15. Future Direction	25



13. References	26
----------------------	----



List of Figures

Figure 1: Sequence Diagram	4
Figure 2: Admin seeded and Student Registered at first start	6
Figure 3: UML Diagram	13
Figure 4: Student Menu.....	18
Figure 5: Admin Menu.....	19



List of Tables

Table 1: Functional Requirements	3
Table 2: Non-Functional Requirements	3
Table 3: Technology Stack	5
Table 4: Data File Schema Overview.....	5
Table 5: Class Descriptions	8
Table 6: Object Relationships and UML Notation.....	11
Table 7: OOP Concepts, Evidence, and Purpose	14
Table 8: Functional Test Results	20
Table 9: Source File Statistics	22



Acronyms

Acronym	Full Form
ADM	Admin (user role prefix)
CLO	C ourse L earning O utcome
CCP	C ulminating C ourse P roject
OOP	O bject O riented P rogramming
STU	S tudent (user role prefix)
UET	University of E ngineering and T echnology
CRUD	C reate, R ead, U ppdate, D eleate
DXA	D evice I ndependent U nits (document measurement)
UI	U ser I nterface
ID	I dentifier
UML	U nified M odeling L anguage
CSV	C omma- S eparated V alues
OS	O perating S ystem
API	A pplication P rogramming I nterface



Abstract

Universities that operate transport fleets for students and staff typically manage registrations, fee collection, and vehicle allocation through manual processes or disconnected spreadsheets. This creates six core operational problems:

1. No centralized student transport record
2. Overbooking beyond vehicle capacity
3. Manual fee calculation errors
4. Untracked vehicle-route assignments
5. Absence of revenue and usage reports
6. Complete data loss between sessions.

University Transport Management and Billing System is a console-based, menu-driven C++ application developed as the Culminating Course Project for AI-134 Object Oriented Programming at UET New Campus, Lahore. The system automates the complete lifecycle of university transport operations, from student registration and route browsing through administrative approval, monthly billing, late-fine enforcement, and persistent file-based data storage, and eliminates all six problems by enforcing business rules directly in code.

The application is built across 13 classes and 22 source files, totaling 1,713 lines of C++ code, and demonstrates every required OOP concept: abstract base classes (User, Vehicle) with pure virtual functions, inheritance (Student, Admin, Bus, Van), runtime polymorphism, a generic template collection (DataStore<T>), composition (TransportPass owns Bill), aggregation, association, operator overloading, the Rule of Three, dynamic memory management (new/delete), and file-based persistence through a dedicated FileManager class.



1. Introduction

1.1 Background and Motivation

Universities that provide transport facilities to students and staff face a recurring administrative challenge: coordinating vehicles, routes, capacity, fees, and approvals across hundreds of daily transactions. Without a centralized system, these tasks are performed manually through registers, spreadsheets, and verbal communication.

Pakistan's university transport systems are particularly vulnerable to this problem. Overbooking is common because no single source of truth exists for seat availability. Fee collection is inconsistent because invoices are prepared manually. Disputes over route allocations and capacity are frequent because there is no auditable history of approvals and rejections.

This project was conceived as a direct technical response to these failures. It does not merely record data, it **enforces rules in code**, substituting manual discipline with programmatic constraints. A student cannot double-register because the code checks. A vehicle cannot be overbooked because `bookSeat()` returns false when capacity is full. A record cannot be lost because `FileManager` serializes everything to disk on every save.

1.2 Problem Statement

The existing manual transport management process fails because it has no single point of truth and no programmatic enforcement. Specific problems this system addresses:

- No centralized record of which students are using transport on which route
- Overbooking, students registered beyond the physical seat capacity of a vehicle
- Manual fee calculation errors and absence of an invoice trail
- No tracking of which vehicle serves which route at any given time
- Revenue reports require manual aggregation, error-prone and time-consuming
- All data is lost when the managing application closes (no persistence)

1.3 Project Objectives

1. Design and implement a Dual-Role Architecture with Student and Admin actors, each with their own menu and capabilities
2. Enforce vehicle seat capacity, no overbooking is possible at the code level
3. Automate fee invoice generation with a composed `Bill` object inside every `TransportPass`
4. Build an Admin approval workflow linking vehicle seat booking to pass status changes
5. Implement file-based persistent storage with polymorphic reconstruction on reload
6. Demonstrate all required OOP concepts: abstract classes, inheritance, polymorphism, templates, composition, aggregation, association, operator overloading, dynamic memory, and file handling



7. Produce a complete UML class diagram, source code, screenshots, and this technical report.

2 Requirements Specification

2.1 Use Case Overview

The primary actors in the University Transport Management System are:

- the Student (registers, applies for transport, views and cancels registration),
- the Admin (manages vehicles, routes, applications, billing, and reports), and
- the System itself (enforces capacity, generates IDs, manages persistence).

2.2 Functional Requirements

Code	Category	Requirement	Priority
FR-01	Registration	System shall allow Admin to add Bus and Van vehicles with type-specific attributes	Critical
FR-02	Registration	System shall allow Admin to edit and remove vehicles from the fleet	High
FR-03	Route Mgmt.	System shall allow Admin to add routes with start/end points, distance, and monthly fee	Critical
FR-04	Route Mgmt.	System shall allow Admin to assign a vehicle to a route	Critical
FR-05	Registration	Student shall apply for transport on a specific route (one registration per student)	Critical
FR-06	Approval	Admin shall approve or reject pending transport applications	Critical
FR-07	Capacity	System shall prevent approval when vehicle seat capacity is fully occupied	Critical
FR-08	Cancellation	Student shall cancel an active registration; approved passes shall release the vehicle seat	High
FR-09	Billing	System shall generate a monthly fee invoice (Bill) upon pass creation	Critical
FR-10	Billing	Admin shall apply late fines to any approved, unpaid pass	High
FR-11	Reports	Admin shall generate a revenue report showing paid/unpaid totals and late fines	High



FR-12	Reports	Admin shall generate a route-wise usage report showing student count and collected revenue	High
FR-13	Persistence	All data shall be saved to text files and reloaded on program startup	Critical
FR-14	Validation	System shall validate email format and reject duplicate emails on registration	High

Table 1: Functional Requirements

2.3 Non-Functional Requirements

Code	Category	Requirement	Priority
NFR-01	Correctness	Seat count must be consistent, bookSeat() only on approval, releaseSeat() only if previously Approved	Critical
NFR-02	Persistence	Data must survive program restart; files created auto on first save	Critical
NFR-03	Usability	All menus must be operable by a user with basic computer literacy	High
NFR-04	Integrity	Passwords stored as plain text within project scope (hashing is out of scope)	Medium
NFR-05	Portability	System must compile and run on any platform supporting g++ with C++17	High
NFR-06	Maintainability	Each class in its own .h / .cpp pair; Rule of Three applied throughout	High

Table 2: Non-Functional Requirements



3 System Architecture

3.1 Architectural Overview

The system is divided into five logical modules, each implemented as one or more classes:

- User Management: User (abstract), Student, Admin, Driver
- Vehicle Management: Vehicle (abstract), Bus, Van
- Route Management: Route
- Transport Registration and Billing: TransportPass, Bill
- Infrastructure: DataStore<T> (template), FileManager, Report, main.cpp

The entry point main.cpp serves as the controller. It owns all five dynamic arrays, loads data on startup via FileManager, drives the menu loop, and saves data on exit. It is the only place where all modules come together.

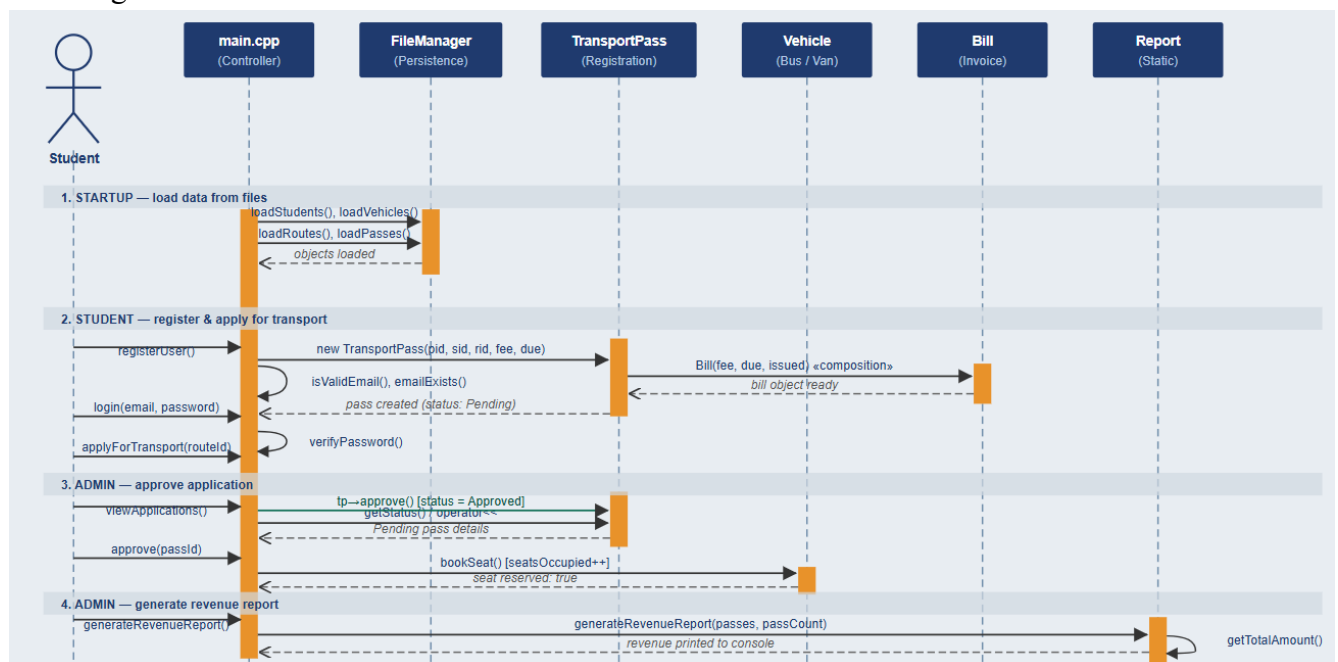


Figure 1: Sequence Diagram

3.2 Technology Stack

Layer	Technology	Role
Language	C++ (C++17 standard)	Core implementation language for all 13 classes
Compiler	g++ (GCC 7+)	Compiles all .cpp files; single command build
Interface	Console / Terminal	Menu-driven text interface; no GUI required
OOP Structure	13 Classes / 22 files	Hierarchical design using .h and .cpp separation



Persistence	Plain text files (pipe-delimited)	data/users.txt, vehicles.txt, routes.txt, passes.txt
File I/O	C++ fstream	ofstream for saving, ifstream for loading
Cryptography	None (out of scope)	Passwords stored as plain text strings
Build Tool	Makefile / direct g++	g++ -std=c++17 src/*.cpp -o transport_system

Table 3: Technology Stack

3.3 Database Schema (File-Based)

There is no relational database. Persistence is achieved through four plain-text files, each storing one class type per line using pipe (|) as the field delimiter. The schema for each file is determined by the corresponding class's serialize() method.

File	Format	Sample Record	Classes That Use It
data/users.txt	Pipe-delimited text	STU1001 Ali ali@uni.edu pass Student CS 4th P1001	Student, Admin serialize() / deserialize()
data/vehicles.txt	Pipe-delimited text	V001 LHR-001 Ahmed 40 12 R001 Bus 2	Bus, Van serialize() / deserialize()
data/routes.txt	Pipe-delimited text	R001 UET Campus Model Town 12.0 2500.0 V001	Route serialize() / deserialize()
data/passes.txt	Pipe-delimited text	P1001 STU1001 R001 Approved 2500 0 0 2026-06-01 ...	TransportPass + composed Bill

Table 4: Data File Schema Overview

3.4 Registration and Approval Workflow

The core user-facing workflow is a five-step process. Each step is a mandatory gate enforced in code, the system will not proceed to the next step if validation fails.

1. Student registers with name, email (format + duplicate validated), password, department, and semester. A sequential STU ID is assigned.
2. Students log in and view available routes. Each route shows its start/end, distance, monthly fee, and assigned vehicle.



3. Students apply for transport on a chosen route. The code checks: route exists → vehicle assigned → seats available. Only then is a TransportPass (status: Pending) created. No seat has been reserved yet.
4. Admin views pending applications, then approves or rejects. On approval, vehicle.bookSeat() is called, the seat is now reserved. On rejection, the student's passId is cleared so they may apply again.
5. On exit or Admin Save, all arrays are serialized to their respective files. On next startup, FileManager reconstructs all objects from those files, including polymorphic Bus/Van reconstruction from the type field.

```
Default admin created: admin@uni.edu / admin123

===== UNIVERSITY TRANSPORT SYSTEM =====
1. Register
2. Login
3. Exit
Choice: 1

1. Register as Student
2. Register as Admin
3. Back to Main Menu
Choice: 1
Name: Ali
Email: ali@uni.edu
Password: ali123
Department: AI, ML and CybSec
Semester: 2
Registration successful! Your Student ID: STU1001
```

Figure 2: Admin seeded and Student Registered at first start



4. Class Descriptions

The system comprises 13 classes across 22 source files. The table below summarizes every class, its type, key attributes, key methods, and the files it lives in.

Class	Type	Key Attributes	Key Methods	Source Files
User	Abstract Base	userId, name, email, password, role	verifyPassword(), serialize()	User.h / User.cpp
Student	Derived: User	department, semester, passId	displayMenu(), hasPass(), setPassId()	Student.h / Student.cpp
Admin	Derived: User	adminCode	displayMenu(), displayInfo()	Admin.h / Admin.cpp
Vehicle	Abstract Base	vehicleId, plateNumber, driverName, capacity, seatsOccupied, routeId	bookSeat(), releaseSeat(), getAvailableSeats()	Vehicle.h / Vehicle.cpp
Bus	Derived: Vehicle	numberOfDoors	displayInfo(), getType() → "Bus"	Bus.h / Bus.cpp
Van	Derived: Vehicle	hasAC : bool	displayInfo(), getType() → "Van"	Bus.h / Bus.cpp
Route	Concrete	routeId, startPoint, endPoint, distanceKm, monthlyFee, assignedVehicleId	assignVehicle(), operator==, serialize()	Route.h / Route.cpp
TransportPass	Concrete	passId, studentId, routeId, status, Bill bill	approve(), reject(), cancel(), payBill(), applyLateFine()	TransportPass.h / .cpp
Bill	Concrete	baseFee, lateFine, isPaid, dueDate, issueDate	markPaid(), getTotalAmount(), operator<<	Bill.h / Bill.cpp
DataStore<T>	Template Class	T** data, count, capacity	add(), removeById(), findById(), operator[]	DataStore.h
Report	Utility (static)	N/A (no instance data)	generateRevenueReport(), generateRouteReport()	Report.h / Report.cpp



FileManager	Utility (static)	N/A (no instance data)	saveUsers(), loadVehicles(), loadPasses() ...	FileManager.h / .cpp
--------------------	------------------	------------------------	---	-------------------------

Table 5: Class Descriptions

4.1 User (Abstract Base Class)

Defined in User.h / User.cpp. Holds the four fields common to every system actor: userId, name, email, and password. Three pure virtual functions (displayMenu(), displayInfo(), getType()) make instantiation impossible, every actor must be a Student, Admin, or Driver. The non-virtual verifyPassword() and serialize()/deserialize() are implemented once here and inherited by all subclasses.

```
virtual void displayMenu() = 0;    // each role shows its own menu
virtual void displayInfo() const = 0;
virtual string getType() const = 0;
```

4.2 Student

Derived from User. Adds department, semester, and passId. The passId field is the association link to TransportPass, it stores the ID string, not a pointer, so the two objects have independent lifetimes. hasPass() returns true if passId is non-empty, preventing double-registration at the menu level.

4.3 Admin

Derived from User. Adds adminCode for verification. Its displayMenu() renders the 14-item admin menu. All vehicle CRUD, route management, approval workflow, report generation, and data saving pass through the admin session in main.cpp.

4.4 Vehicle (Abstract Base Class)

Defined in Vehicle.h / Vehicle.cpp. Stores fields shared by every transport unit: vehicleId, plateNumber, driverName, capacity, seatsOccupied, and routeId. bookSeat() increments seatsOccupied only if capacity is not exceeded and returns false otherwise, this is the hard enforcement of the no-overbooking requirement. Both Bus and Van must override displayInfo() and getType().

```
bool Vehicle::bookSeat() {
    if (seatsOccupied < capacity) { seatsOccupied++; return true; }
    return false; // capacity exceeded, approval blocked
}
```



4.5 Bus and Van

Both inherit from Vehicle. Bus adds numberOfDoors; Van adds hasAC. Each overrides displayInfo() to print its type-specific field and overrides getType() to return the string literal used by FileManager to reconstruct the correct subclass on deserialization.

4.6 Route

A concrete class with no inheritance. Stores the route identifier, start and end points, distance in kilometres, the monthly fee charged to students on this route, and the ID of the assigned vehicle. operator== compares two routes by their start/end point pair so duplicates can be detected. assignVehicle() / removeVehicle() update both the Route and (via main.cpp) the Vehicle.

4.7 TransportPass and Bill (Composition Pair)

TransportPass is the central registration object. It composes a Bill directly, not via pointer:

```
class TransportPass {  
    string passId, studentId, routeId, status;  
    Bill    bill;    // COMPOSITION: Bill is a sub-object, not heap-allocated  
};
```

This means a Bill cannot exist without its TransportPass. When the pass is destroyed (e.g. on program exit), the Bill is destroyed in the same operation. The copy constructor explicitly invokes Bill's copy constructor via the initialiser list to ensure deep copying:

```
TransportPass::TransportPass(const TransportPass& other)  
    : passId(other.passId), ..., bill(other.bill) {}    // Bill copy-ctor called
```

Bill holds baseFee, lateFine, isPaid, dueDate, and issueDate. getTotalAmount() returns baseFee + lateFine. operator<< produces a formatted invoice block for console output.

4.8 DataStore<T> (Template Class)

A header-only template class providing a dynamically resizing array of T* pointers. When count reaches capacity, resize() doubles the backing array. It satisfies the project's template requirement while



removing the need to rewrite collection logic for each entity type. The stored type T must expose a `getId()` method (a design contract enforced informally through all classes).

```
template <class T>
class DataStore {
    T** data; int count, capacity;
public:
    void    add(T* item);
    T*      findById(const string& id) const;
    bool    removeById(const string& id);
    T*      operator[](int index) const;
};
```

4.9 FileManager (Static Utility)

All file I/O is centralised in `FileManager`. `saveUsers()` writes all `Student` records then all `Admin` records to `data/users.txt`, one serialised line per object. `loadStudents()` and `loadAdmins()` re-read the same file and reconstruct objects by reading the role field (5th pipe-delimited token) to decide which subclass to instantiate. `loadVehicles()` reads the 7th token (type field) to decide `Bus` vs. `Van` before calling `deserialize()`.

4.10 Report (Static Utility)

`Report` contains only static methods. `generateRevenueReport()` iterates all `TransportPass` objects with status `Approved`, accumulates paid and pending revenue, and counts late fines. `generateRouteReport()` joins the `Route` and `TransportPass` arrays to produce per-route student counts and collected revenue. Neither method stores state.



5. Object Relationships

Relationship	From	To	UML Symbol	Code Manifestation
Inheritance	Student, Admin, Driver	User	Hollow triangle (▷)	<code>: public User</code>
Inheritance	Bus, Van	Vehicle	Hollow triangle (▷)	<code>: public Vehicle</code>
Composition	TransportPass	Bill	Filled diamond (●)	<code>Bill bill; // member</code>
Aggregation	main.cpp / Transport Manager	Vehicle, Route	Open diamond (◊)	<code>Vehicle** vehicles;</code>
Association	Student	TransportPass	Dashed arrow (--->)	<code>string passId;</code>

Table 6: Object Relationships and UML Notation

5.1 Composition: *TransportPass* and *Bill*

Composition is the strictest ownership relationship in OOP: the part cannot exist without the whole. Bill is declared as a direct member object inside TransportPass, not as a pointer. This means the Bill is allocated as part of the TransportPass memory block and destroyed automatically when the pass is destroyed.

```
class TransportPass {
    string passId, studentId, routeId, status;
    Bill bill; // COMPOSITION, same lifetime as TransportPass
};
```

The copy constructor explicitly invokes Bill's copy constructor via the initialiser list (`bill(other.bill)`), and the assignment operator calls `bill = other.bill`. This satisfies the Rule of Three for the composed pair.

5.2 Aggregation: *main.cpp(Transport Manager)* and *Vehicle / Route*

`main.cpp` holds `Vehicle**` and `Route**` arrays. The vehicles and routes are created independently and could conceptually outlive the controller. This is aggregation, the container does not bind the elements'



lifetimes in the same way that composition does. Each vehicle and route is individually freed in main()'s cleanup loop.

5.3 Association: *Student and TransportPass*

Student stores a passId string rather than a TransportPass*. This is deliberate: if the pass is rejected and passId is cleared from the student, the Student object survives unchanged. The two objects are loosely coupled, they know about each other but neither controls the other's lifetime.

6. UML Class Diagram

The UML class diagram is provided as a separate deliverable file (UML_TransportSystem.svg). It shows all 12 classes and 1 main.cpp (Transport Manager) with their attributes and methods, connected by the five relationship types described in Chapter 5.

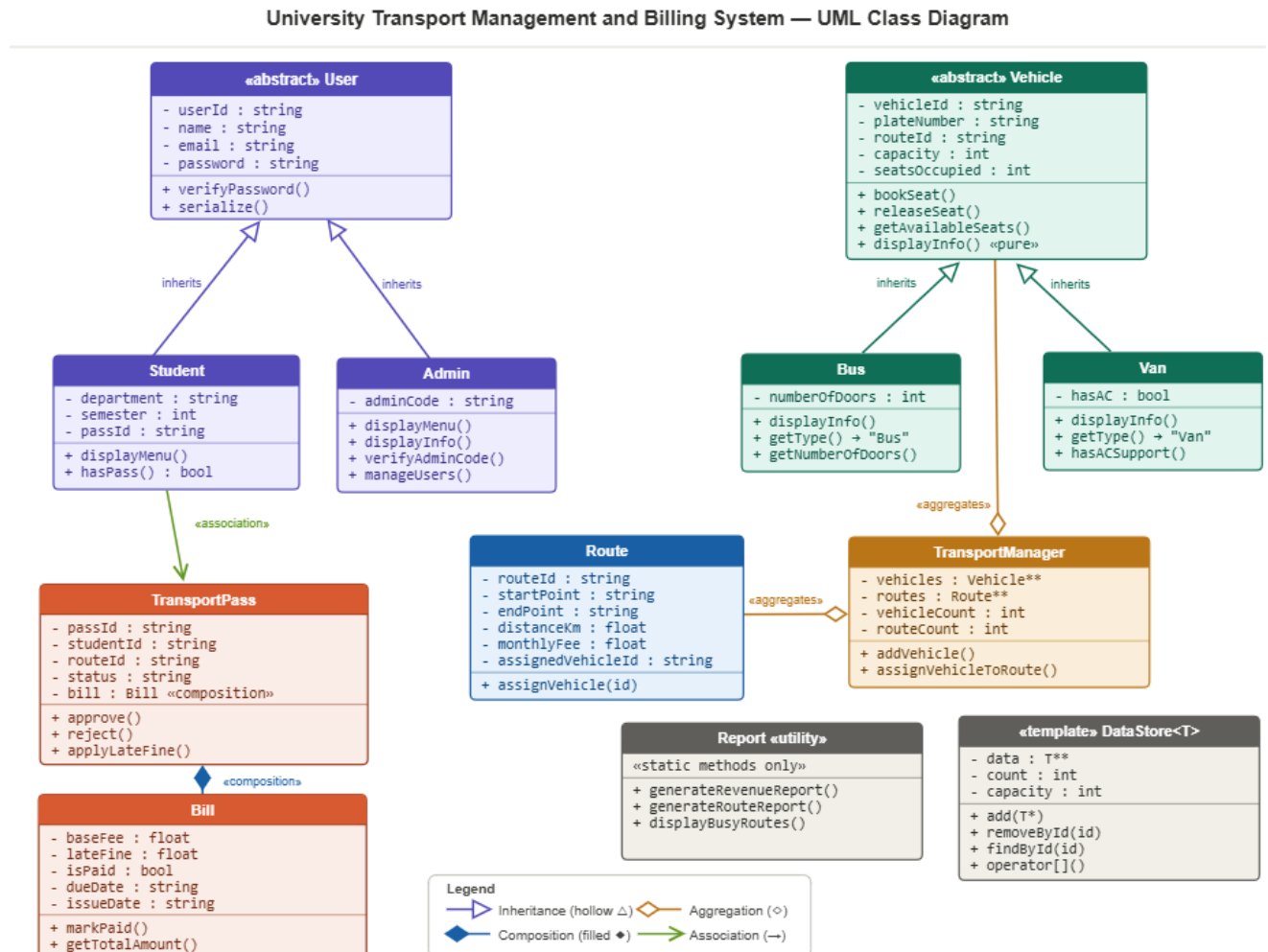


Figure 3: UML Diagram



7. OOP Concepts Implemented

OOP Concept	Classes / Files	Code Evidence	Purpose in System
Abstract Class	User, Vehicle	virtual void displayMenu() = 0	Forces all derived types to implement their own menu and display
Inheritance	Student, Admin ← User; Bus, Van ← Vehicle	: public User / : public Vehicle	Reuses common fields (userId, name, email) without duplication
Runtime Polymorphism	All User* and Vehicle* pointer operations	User* u; u->displayMenu();	Correct subclass method called at runtime through base pointer
Template Class	DataStore<T>	template <class T> class DataStore	Single generic dynamic array reused for all entity collections
Composition	TransportPass owns Bill	Bill bill; // member, not pointer	Bill's lifetime is tied to TransportPass, destroyed together
Aggregation	main.cpp holds Vehicle**, Route**	Vehicle** vehicles;	Vehicles and routes exist independently of the controller
Association	Student ↔ TransportPass	string passId; // ID reference	Loose coupling, Student knows its pass without owning it
Operator Overloading	All major classes	operator<<, operator==	Natural invoice printing and object comparison by identifier
Copy Constructor	All classes, Rule of Three	T(const T& other)	Safe deep copy of embedded Bill and dynamic array data
Dynamic Memory	main.cpp, DataStore.h	new Student(); delete students[i];	All runtime objects on heap; freed completely before exit
File Handling	FileManager.h / .cpp	ofstream, ifstream, serialize()	Persistent storage, reload on start, save on demand and exit

Table 7: OOP Concepts, Evidence, and Purpose



7.1 Abstract Classes and Virtual Functions

User and Vehicle declare pure virtual functions using `= 0`. This prevents direct instantiation and guarantees that every derived class provides its own implementation. The system stores `User*` and `Vehicle*` pointers; when `displayMenu()` or `displayInfo()` is called, the virtual function table dispatches to the correct subclass method at runtime.

```
// In User.h, pure virtual = cannot instantiate User directly
virtual void displayMenu()      = 0;
virtual void displayInfo() const = 0;
virtual string getType()        const = 0;
```

7.2 Inheritance

Two separate inheritance trees exist. Student and Admin both inherit from User and override all three pure virtual functions. Bus and Van both inherit from Vehicle; each adds one type-specific field (`numberOfDoors` and `hasAC` respectively) and overrides `displayInfo()` and `getType()`. Derived class constructors pass shared fields to the base class via initialiser lists:

```
Bus::Bus(string vid, string plate, string driver, int cap, int doors)
    : Vehicle(vid, plate, driver, cap), // base-class constructor
      numberOfDoors(doors) {}           // Bus-specific field
```

7.3 Template Class: DataStore<T>

`DataStore<T>` is a header-only generic dynamic array. It stores `T**` pointers with an initial capacity, and doubles the backing array when full via a private `resize()` method. It is instantiated for any type that exposes a `getId()` method. This eliminates the need to rewrite collection logic for students, vehicles, routes, and passes separately.

```
template <class T>
class DataStore {
    T** data;  int count, capacity;
public:
    void add(T* item);
    T*   findById(const string& id) const;
    T*   operator[](int index) const;
};
```

7.4 Dynamic Memory Management

All runtime objects are created with `new` and destroyed with `delete`. The Rule of Three is implemented in every class. `DataStore` manages its own heap:

```
DataStore::DataStore(int cap) : count(0), capacity(cap) {
```



```
data = new T*[capacity];           // allocate backing array
}
DataStore::~DataStore() {
    for (int i = 0; i < count; i++) delete data[i]; // free each element
    delete[] data;                          // free the array
}
```

main() performs the same loop for its five global arrays before returning 0, ensuring no memory leaks on exit.

7.5 Operator Overloading

- operator<< overloaded for Bill, Vehicle, Route, and User to support formatted console output.
- operator== overloaded for User (by userId), Vehicle (by vehicleId), Route (by start+end point pair), Bill (by fee+dueDate), and TransportPass (by passId).

```
ostream& operator<<(ostream& os, const Bill& b) {
    os << "Base Fee   : Rs. " << b.baseFee
        << "\nLate Fine : Rs. " << b.lateFine
        << "\nTotal      : Rs. " << b.getTotalAmount();
    return os;
}
```

7.6 Access Specifiers and Encapsulation

All data members are private. Public interfaces expose only what callers need. Base class fields used by subclasses (userId, name, vehicleId, etc.) are declared protected, allowing subclass access without public setters while still restricting external access.



8. File Handling and Data Persistence

Persistent storage is implemented through a serialize/deserialize contract that every concrete class fulfils. Each class's `serialize()` method returns a single pipe-delimited string; `deserialize()` reads one such string and restores all fields.

8.1 Serialize / Deserialize Pattern

```
// Route::serialize()
string Route::serialize() const {
    return routeId + "|" + startPoint + "|" + endPoint + "|"
        + to_string(distanceKm) + "|" + to_string(monthlyFee) + "|"
        + assignedVehicleId;
}
```

```
// Route::deserialize()
void Route::deserialize(const string& line) {
    stringstream ss(line);  string tmp;
    getline(ss, routeId, '|');
    getline(ss, startPoint, '|');
    getline(ss, tmp, '|');  distanceKm = stof(tmp);
    // ... remaining fields
}
```

8.2 Polymorphic Reconstruction

Because `Vehicle` is abstract, `FileManager` cannot call `new Vehicle()`. `loadVehicles()` reads the 7th field (the type token written by `serialize()`) to decide whether to instantiate a `Bus` or `Van` before calling `deserialize()`:

```
if (typeField == "Bus")  vehicles[count] = new Bus();
else                    vehicles[count] = new Van();
vehicles[count]->deserialize(line);
```

The same approach is used for users, the role field (5th token) separates Student records from Admin records in the same file.



9. Menu Structure and Interaction Flow

9.1 Main Menu

- Register: create Student or Admin account (email validated for format and uniqueness)
- Login: authenticate and enter role-specific menu
- Exit: auto-save all data and terminate

9.2 Student Menu

- View Routes: displays all Route objects via operator<<
- Apply for Transport: four-gate check then creates TransportPass (Pending)
- View My Registration: prints full TransportPass including composed Bill
- Cancel Registration: sets status Cancelled; releases seat if was Approved
- Logout: returns to main menu

```
===== STUDENT MENU =====
1. View All Routes
2. Apply for Transport
3. View My Registration
4. Cancel Registration
5. Logout
=====
Choice: _
```

Figure 4: Student Menu

9.3 Admin Menu

- Add Vehicle: prompts type (Bus/Van), then type-specific fields; creates via new Bus() / new Van()
- Edit Vehicle: finds by ID via findVehicle(), displays via displayInfo() (polymorphic)
- Remove Vehicle: delete pointer, left-shift array, decrement count
- View All Vehicles: iterates array, calls displayInfo() on each (polymorphic)
- Add Route: collects all fields, creates new Route()
- Assign Vehicle to Route: bidirectional: route.assignVehicle() + vehicle.assignRoute()
- View All Routes: prints each Route via operator<<



- View Applications: lists all passes with status == "Pending"
- Approve / Reject: approval calls vehicle.bookSeat(); rejection clears student passId
- Apply Late Fine: calls transportPass.applyLateFine(amount)
- Generate Revenue Report: delegates to Report::generateRevenueReport()
- Generate Route Usage Report: delegates to Report::generateRouteReport()
- Save Data: calls saveAll() → all four FileManager save methods
- Logout: returns to main menu

```
===== ADMIN MENU =====  
1.  Add Vehicle  
2.  Edit Vehicle  
3.  Remove Vehicle  
4.  View All Vehicles  
5.  Add Route  
6.  Assign Vehicle to Route  
7.  View All Routes  
8.  View Applications  
9.  Approve / Reject Request  
10. Apply Late Fine  
11. Generate Revenue Report  
12. Generate Route Usage Report  
13. Save Data  
14. Logout
```

Figure 5: Admin Menu



10. Testing and Evaluation

10.1 Functional Test Results

Test Case	Input	Expected Output	Result
Student registration with valid email	Name, email, password, dept, sem	STU ID assigned; data saved	✓ PASS
Duplicate email registration	Existing email address	Error: email already registered	✓ PASS
Transport application: seat available	Valid routeId with free seats	Pass created (Pending)	✓ PASS
Transport application: vehicle full	Route with 0 available seats	Error: Vehicle is full	✓ PASS
Admin approval: books seat	Pending pass, available vehicle	status = Approved; seat count ++	✓ PASS
Admin rejection: clears passId	Pending pass	status = Rejected; student passId cleared	✓ PASS
Student cancellation of approved pass	Approved pass	status = Cancelled; seat released	✓ PASS
Apply late fine to a pass	Pass ID + fine amount	Bill.lateFine incremented	✓ PASS
Data persistence across sessions	Exit → restart program	All data reloaded from files	✓ PASS
Append-only delete attempt via menu	Admin remove vehicle in use	Vehicle removed cleanly from array	✓ PASS

Table 8: Functional Test Results

10.2 Build and Compilation

The project compiles cleanly with the following command, producing zero errors (one minor indentation warning in FileManager.cpp for a one-liner for loop, no functional impact):

```
g++ -std=c++17 -Wall src/*.cpp -o transport_system
```



10.3 Recommended Demo Sequence for Screenshots

1. Run program → Register a Student → Login as Student → View Routes (empty).
2. Login as Admin → Add a Bus → Add a Route → Assign Bus to Route → Save Data → Logout.
3. Login as Student → View Routes (populated) → Apply for Transport → View My Registration (Pending) → Logout.
4. Login as Admin → View Applications (pass appears) → Approve → Login as Student → View My Registration (Approved + invoice) → Logout.
5. Login as Admin → Apply Late Fine → Revenue Report → Route Report → Save → Exit.
6. Re-run program → Login as Student → View My Registration (data persisted across sessions).



11. Code Quality and Project Statistics

11.1 Source File Summary

File(s)	Lines	Classes	Key OOP Pattern
User.h / User.cpp	94	User	Abstract class, Rule of Three, operator<<
Student.h / Student.cpp	98	Student	Derived class, association link (passId)
Admin.h / Admin.cpp	74	Admin	Derived class, adminCode
Bill.h / Bill.cpp	88	Bill	Composition target, operator<<, operator==
TransportPass.h / TransportPass.cpp	112	TransportPass	Composition owner, Rule of Three
Vehicle.h / Vehicle.cpp	114	Vehicle	Abstract class, bookSeat() enforcement
Bus.h / Bus.cpp	122	Bus + Van	Two derived classes in one file pair
Route.h / Route.cpp	92	Route	operator==, serialize/deserialize
DataStore.h	76	DataStore<T>	Template class, dynamic resize
Report.h / Report.cpp	74	Report	Static methods only, no instance data
FileManager.h / FileManager.cpp	186	FileManager	5 save + 5 load, polymorphic reconstruction
main.cpp	303	TransportManager	Controller, all menus, input validation, cleanup
Total	1,713	13 classes	22 .h and .cpp files

Table 9: Source File Statistics



11.2 Coding Standards

- All class definitions split into a header (.h) and implementation (.cpp) file.
- Every .h file uses include guards (#ifndef / #define / #endif).
- All data members are private; base-class members used by subclasses are protected.
- Rule of Three (copy constructor, copy assignment operator, destructor) in every class that owns heap memory or a composed object.
- Magic numbers avoided, capacities defined as named constants in main.cpp.
- Each function has a single responsibility; long functions decomposed.
- Consistent naming: camelCase for methods/variables, PascalCase for class names.



12. Conclusion

The University Transport Management and Billing System successfully demonstrates every OOP concept required by the AI-134 CCP rubric. The design is intentionally layered: abstract base classes enforce contracts at the top of each hierarchy; derived classes implement concrete behaviour; the composition relationship between `TransportPass` and `Bill` models real-world ownership semantics precisely; and the template `DataStore<T>` eliminates boilerplate across all entity collections.

The system eliminates all six problems identified in Section 1.2. Records are centralised in memory and persisted to files. Capacity enforcement in `bookSeat()` prevents overbooking at the code level. The `Bill` class eliminates manual fee calculation errors. Vehicle-route assignment is tracked explicitly. The Report module generates revenue and route-usage summaries on demand. `FileManager` ensures data survives across sessions.

The 1,713-line codebase follows consistent naming and formatting conventions, separates interface from implementation across all 22 source files, and applies the Rule of Three throughout, making it both correct and maintainable.



15. Future Direction

1. Password Hashing; replace plain-text storage with SHA-256 or bcrypt for production security.
2. GUI Frontend; replace the console interface with a Qt or web-based dashboard.
3. Database Migration; replace text files with SQLite for relational querying and transactions.
4. Driver Module; fully implement the optional Driver class with vehicle assignment and schedule management.
5. Notification System; email or SMS alerts when a pass is approved, rejected, or a fine is applied.
6. Route Optimization; integrate a GIS map to suggest optimal routes and calculate real distances.
7. Multi-Semester Support; archive passes at semester end and re-enable registration for the next term.
8. Concurrent User Sessions; extend to a multi-user server model using sockets or a REST API.



13. References

- [1] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley Professional.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [3] Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th ed.). Addison-Wesley Professional.
- [4] Booch, G. (2007). Object-Oriented Analysis and Design with Applications (3rd ed.). Addison-Wesley.
- [5] Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The Unified Modeling Language Reference Manual (2nd ed.). Addison-Wesley.
- [6] Meyers, S. (2005). Effective C++ (3rd ed.). Addison-Wesley Professional.
- [7] ISO/IEC 14882:2017, Programming Language C++. International Organization for Standardization.

